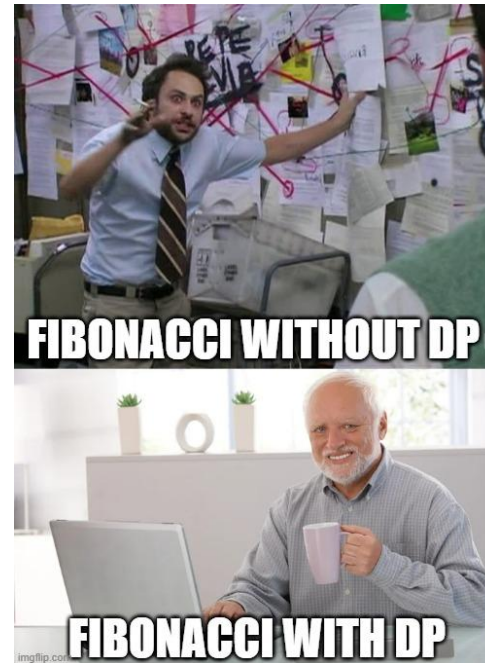


D3: Recurrence Relations & Dynamic Programming



Follow along with the discussion worksheet!



Sec 19: Th 4:30-5:30pm 2153 GGBL
IA: Anthony Chalhoub

Agenda

- ▶ Recurrence Relations
- ▶ Dynamic Programming



Recurrence Relations

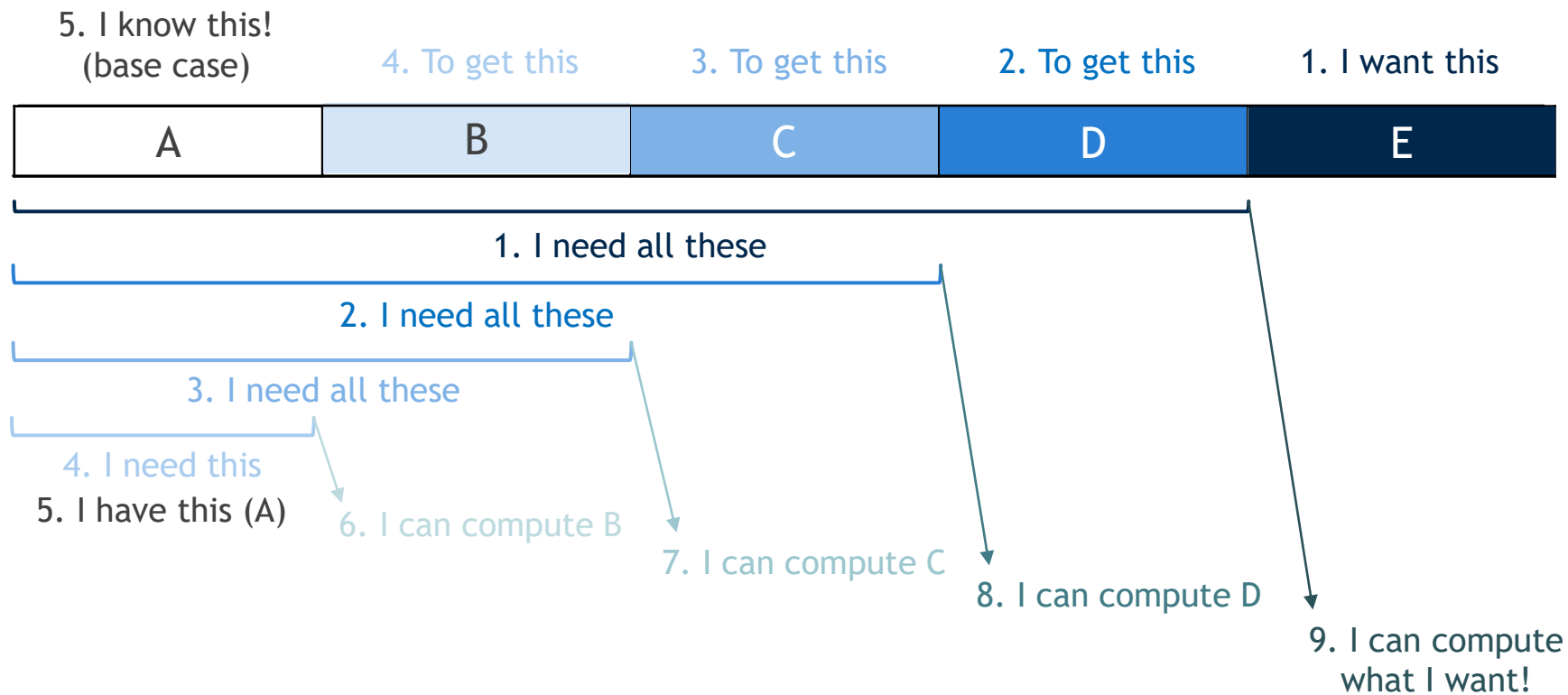


Recurrence Relations

- ▶ For now, put aside everything you know about (memoization, bottom-up table filling, ...)
- ▶ From EECS 203:
 - ▶ $T(n)$ is defined in terms of **previous values**, i.e., $T(k)$ where $k < n$
 - ▶ We need initial values of T to fully define the recurrence (aka base cases)
- ▶ **Example:** Define a recurrence relation (including base cases) to the following problem:
"How many ways can you climb n stairs if you can take 1, 2, or 3 steps at a time?" (WS #1)

Recurrence Relations and Dynamic Programming

- ▶ Dynamic programming is a powerful technique to handle problems with **overlapping subproblems**
- ▶ **Recurrence relations** captures the dependencies between these subproblems- giving us a starting point of approaching a DP problem



Recipe for DP Recurrence Relations

► Step 1: Identify the ‘value version’ of the problem

- **VERY IMPORTANT-** Start your solution with “Let [name of recurrence] be [definition of recurrence]”
- Puts aside the question of finding an optimal object itself, and instead just seeks its value
- Sometimes, you may need to transform the problem to overlap subproblems
- **Example (LIS):** Let $\text{END-LIS}(i)$ be the length of longest increasing subsequence ending at $A[i]$

► Step 2: Identify the base case(s)

- Which value do you know without relying to solutions from other subproblems?
- This should give you a sense of what direction to fill the table in later
- **LIS:** $\text{END-LIS}(i) = 0$ if $i = 0$

Recipe for DP Recurrence Relations, cont.

► Step 3: Identify subproblem dependencies

- Determine the **criteria(s)** under which the problem depends on smaller subproblems, and **how to reduce** it to its corresponding subproblem(s) (e.g., shrinking an index, narrowing a range, ancestor vertices, etc.)
 - **LIS:** If $j < i$ and $A[j] < A[i]$, then we could reduce $\text{END-LIS}(i)$ to $1 + \text{END-LIS}(j)$
- Express solution for **current problem** as **solution(s) from dependent subproblems**
 - **LIS:** $\text{END-LIS}(j) = 1 + \max\{\text{END-LIS}(j): j < i \text{ and } A[j] < A[i]\}$

If there are multiple values contributing to a single case, you need to think about how to "combine" them

Useful Notations

Sum Σ / product Π

Over an array
 $A[1, \dots, n]$

$$\sum_{i=1}^n A[i] = A[1] + A[2] + \dots + A[n]$$

$$\prod_{i=1}^n A[i] = A[1] \cdot A[2] \cdot \dots \cdot A[n]$$

Over a set
 $S = \{s_1, \dots, s_n\}$

$$\sum_{s \in S} s \quad \prod_s s$$

Conditional

$$\sum_{i: i \text{ is even}} A[i] \quad \prod_{s \in S: s > 0} s$$

Max/ min

Over an array
 $A[1, \dots, n]$

$$\max_{i: 1 \leq i \leq n} A[i]$$

Over a set
 $S = \{s_1, \dots, s_n\}$

$$\min_{s \in S} s$$

Sets Operations

Union

$$\bigcup_{i=1}^n S_i = S_1 \cup S_2 \cup \dots \cup S_n$$

Intersection

$$\bigcap_{i=1}^n S_i = S_1 \cap S_2 \cap \dots \cap S_n$$

Set minus

$$A \setminus B = A \cap \bar{B}$$

It is common to write $S \leftarrow S \setminus \{s\}$ for
“remove element s from set S ”

Big AND $\wedge$ / big OR $\vee$

Over an array
of truth values
 $T[1, \dots, n]$

$$\bigwedge_{i=1}^n T[i] = T[1] \wedge T[2] \wedge \dots \wedge T[n]$$

$$\bigvee_{i=1}^n T[i] = T[1] \vee T[2] \vee \dots \vee T[n]$$

Tilings (WS #3)

Lambda wants to tile a path of length n using k types of tiles with distinct lengths $\ell_1, \ell_2, \dots, \ell_k$ (assume λ has infinite supply of each type of tile). λ wants to know the total number of ways of tiling of such a path.

Define a recurrence relation (including base case(s)) that are suitable for a dynamic programming solution to the problem.

Hint: Let $T(i)$ = number of ways to tile a path of length i



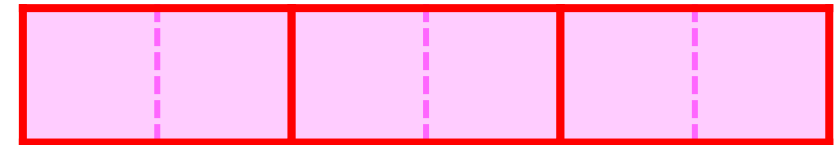
Path:



Tiles:



Possible tilings:



Intermission

10 Tips to Succeed in EECS 376 (Part 2)

3

Keep Up with Lectures

Schedule

Day	#	Material and Readings in Notes	Deadline
Design and Analysis of Algorithms			
Mon 6 Jan		No Class	
Wed 8 Jan	L01	Introduction, The Potential Method	
Discussion	DO1	Review: Proofs, Asymptotic Notation, Induction	
Mon 13 Jan	L02	Divide and Conquer 1	
Wed 15 Jan	L03	Divide and Conquer 2	HW1 8pm ET
Discussion	DO2	Divide and Conquer	
Mon 20 Jan		No Class - MLK Day	
Wed 22 Jan	L04	Dynamic Programming 1	HW2 8pm ET
Discussion	DO3	Divide and Conquer & Dynamic Programming	

To **fully utilize your time in discussion**, you should catch up with lectures of the week before coming.

4

Bookmark Important Slides



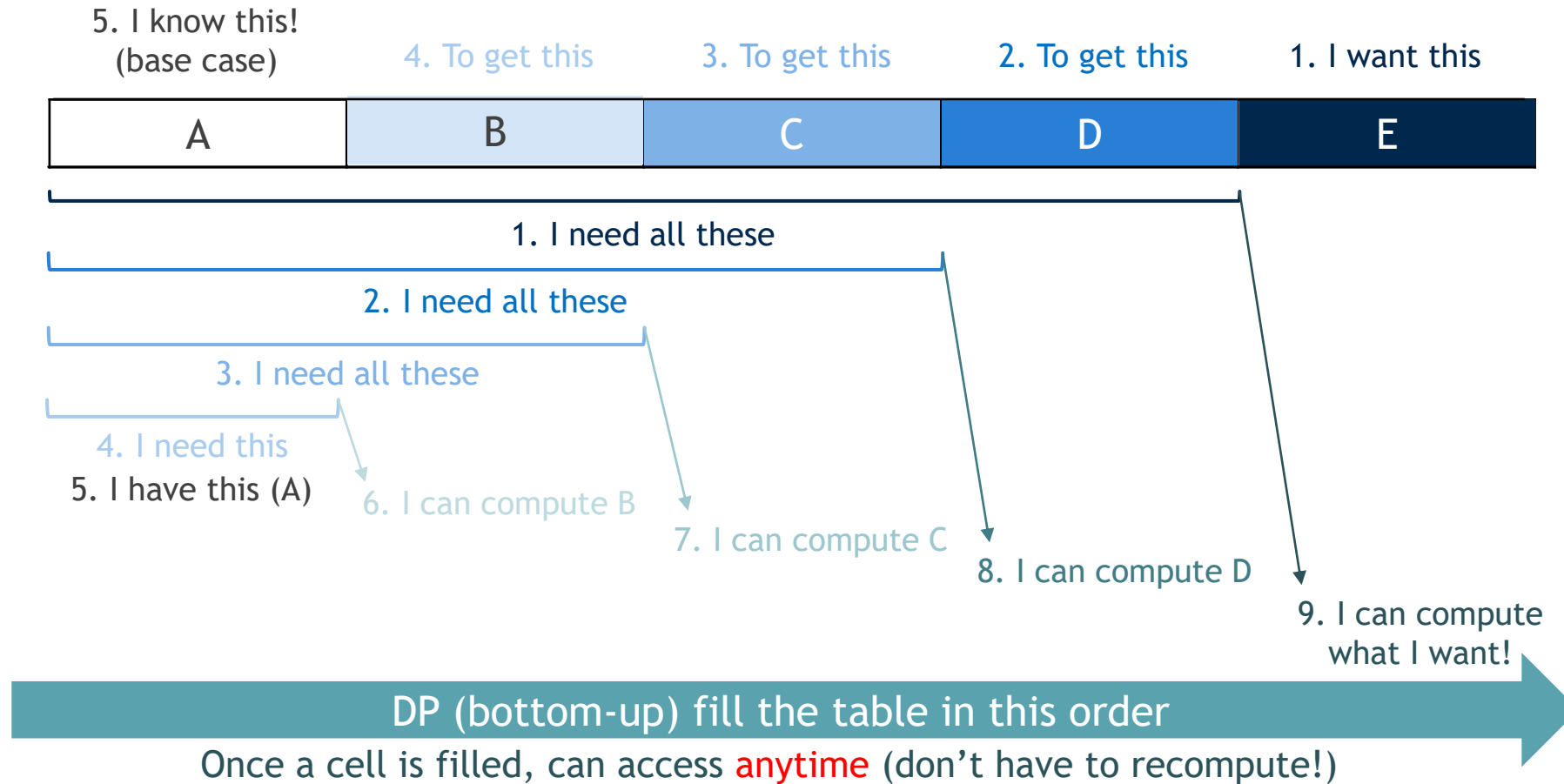
Bookmark important things that you know you want to put on your **cheatsheet** as you see them for the first time- so rather than reading through 500 slides in 2 hours maybe you just have to make your cheatsheet from 50 important slides.

Dynamic Programming



Dynamic Programming

- Re: DP is powerful when the problem can be reduced into **overlapping subproblems**



Transformed Problem and Final Answer

- Recall that in the LIS problem, instead of solving it directly, we transformed the problem into $\text{END-LIS}(i) = \text{LIS ending at } A[i]$

- Rationale:** To overlap subproblems

END-LIS Table

--	--	--	--	--	--	--	--	--

This cell is not the final answer!

$\text{LIS}(A) = \max_i \text{END-LIS}(i) = \text{max element in this table}$

This value is the final answer

Sometimes the final answer could be the min of a column

--	--	--	--	--	--	--	--	--

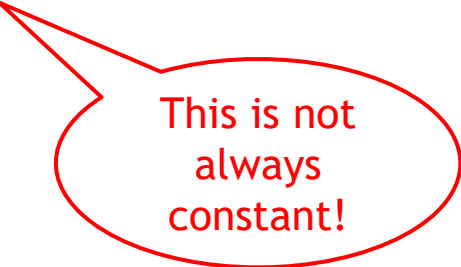
Or a particular cell in the table

This is why an ENGLISH definition for the recurrence relation is important!

DP Implementation & Runtime Analysis

- ▶ Once we have the recurrence relation, the implementation is often *mechanical*
- ▶ Things to take note on:
 - ▶ **Table dimension:** Determine the dimensionality of the table ← Will talk about this next week
 - ▶ **Dependencies:** Identify which cells must be filled **before** others
 - ▶ **Final answer:** Know which cell holds the **final solution** to the original problem
- ▶ The runtime analysis of a DP solution *typically* looks like

Number of cells $\times$ Cost to fill in a cell



This is not
always
constant!

Back to Tilings...

- ▶ $T(i)$ = number of ways to tile a path of length i

$$T(i) = \begin{cases} 1 & \text{if } i = 0 \\ \sum_{j=1}^k T(i - \ell_j) & \text{if } i > 0 \text{ and } i - \ell_j \geq 0 \end{cases}$$

- ▶ Final answer: $T(n)$
- ▶ We can convert it into a DP algorithm as follows:

Upsilon Time!

```
function TILINGS( $n, \{\ell_1, \dots, \ell_k\}$ ):
```

```
    Initialize an empty table of size  $n$ 
```

```
    table[0]  $\leftarrow$  1 // Base case
```

```
    for  $i = 1, \dots, n$  do
```

```
        table[ $i$ ]  $\leftarrow \sum_{i: 1 \leq j \leq k \text{ and } i - \ell_j \geq 0} \text{table}[i - \ell_j]$ 
```

```
    return table[ $n$ ] // Final answer
```

- Upsilon: There is only one for loop of n iterations, so the runtime of the algorithm is $O(n)$

The Full DP Recipe

- ▶ Step 0: Determine suitable dimensionality for the problem ← Will talk about this next week

// Recurrence relation- Math only, no data structures

- ▶ Step 1: Identify the value version of the problem
- ▶ Step 2: Identify the base case(s)
- ▶ Step 3: Identify subproblem dependencies

// Actual implementation- Think like a programmer

- ▶ Step 4: Describe the algorithm (see [Handout 3: Giving an Algorithm](#))
 - ▶ Step 4a: Reconstructing solution (backtracking) ← Will talk about this next week
 - ▶ Step 4b: Correctness Proof
 - ▶ Step 4c: Runtime analysis

Practice!



Longest Arithmetic Subsequence (WS #4)

- ▶ **Definition:** An *arithmetic subsequence* S of an array of integers A is a subsequence such that

$$S[i + 1] = S[i] + d$$

for all $i = 1, \dots, |S|$.

- ▶ **Example:** $A = [1, 2, 3, 4, 5, 6, 7]$, $d = 2$, $\text{LAS}(A) = 4$
- ▶ Given an array of integers A and a difference d , we want to find the length of *any* longest arithmetic subsequence with difference d .
- ▶ Derive a recurrence relation (including base cases) that is suitable for a DP solution to the above problem. Then, give a dynamic programming algorithm that return the length of the LAS
- ▶ **Hint:** Recurrence relation for LIS might give you some ideas.



Bonus Problem!



McNuggets (WS #2)

McNuggets

We have a collection M of chicken McNuggets meals; these meals are displayed to you in a menu, represented as an array $M[1, \dots, n]$, with the number of McNuggets per meal. Your goal is to determine, for a given positive integer t , whether it is possible to purchase *exactly* t McNuggets.¹ For example, for $M = [4, 6, 10, 20]$ and $t = 22$, it is possible with $M[2] + M[2] + M[4] = 22$; however, for the same M and $t = 13$, it is not possible.²

Give a recurrence relation (including base cases), that is suitable for a dynamic programming solution to solve this problem in $O(t \cdot n)$ time. Your solution should include an explanation of why the recurrence is correct.



She gave you her number? She gives me her order, we are not the same bro 🤔



HOW IT'S SAID:



HOW YOU SAY IT:

